MODULE 06 · DOMAIN 1 - DATA PREPARATION FOR ML AND AI

Feature Store

One definition of a feature, served to training and to inference alike.

MODULE 06 of 43 DOMAIN 1 - Data Prep WEIGHT 28%

admin@sthithapragna.com

The same feature, computed twice by two teams, is **two different features**.

Two problems, one service

- **Consistency.** One definition, written once, read by training and by inference.
- **Reuse.** Features are discoverable, so a second team finds them instead of rebuilding.
- The offline store holds **history**. The online store holds the **latest value**, fast.
- Point-in-time queries build training sets without leaking the future.
- Lineage links features to the models trained on them, which matters for audit.

> ON THE EXAM
>
> Consistency is the exam answer. Reuse is the reason organisations actually adopt it.

One write, two readers

01

Compute

A job or stream calculates the feature value.

02

Ingest

PutRecord writes it to the feature group.

03

Split

It lands in the online store, the offline store, or both.

04

Read

Training reads history. The endpoint reads the latest value.

ON THE EXAM

The single write is the whole point. Two readers, one source, no second implementation to drift.

Feature groups, and the two mandatory fields

- A **feature group** is a table: a set of related features sharing an entity.
- **Record identifier** is required. The key, such as customer_id.
- **Event time** is required. When this value became true.
- Event time is what makes history queryable and point-in-time joins possible.
- Group features by entity, not by model. One customer group, not one group per model.

> ON THE EXAM
>
> Event time is the field people treat as bookkeeping. It is the field the whole time-travel capability rests on.

Feature definitions, and the three data types

TYPE	WHAT IT HOLDS
`String`	Text, categories, and anything you will encode later
`Integral`	Whole numbers. Counts, identifiers, flags
`Fractional`	Decimals. Ratios, averages, monetary amounts
`Event time`	String in ISO-8601, or Fractional as a Unix timestamp

ON THE EXAM

Only three types. Booleans become Integral, dates become String or Fractional, and complex objects do not belong here.

Three storage configurations

CONFIGURATION	CHOOSE IT WHEN
Online only	Real-time inference needs the value. History is not needed
Offline only	Training and batch inference only. Nothing serves in real time
Online and offline	The usual production choice. Train on history, serve the latest

ON THE EXAM

Online plus offline is the configuration that actually removes skew, because both readers come from one write.

The online store

- Managed key-value store. **Single-digit millisecond** retrieval by record identifier.
- **It keeps only the latest record** per identifier. It is not a history store.
- The typical pattern: request arrives with an ID, endpoint fetches features, then predicts.
- **TTL** can expire records automatically, which is useful for short-lived session features.
- Reachable privately over PrivateLink through the featurestore-runtime endpoint.

> ON THE EXAM
>
> The endpoint receives an ID rather than fifty features. That alone simplifies the calling application enormously.

Standard tier or InMemory tier

- **Standard** is the default online tier, and it supports online plus offline groups.
- **InMemory** targets the lowest latency and highest throughput.
- **InMemory supports online-only feature groups.** Not online plus offline.
- So there is **no replication to an offline store** on InMemory. No history at all.
- Default InMemory maximum size is **50 GiB**. It scales automatically, and may throttle while it does.

> ON THE EXAM
>
> You cannot have InMemory latency and an offline history store in the same feature group. That is a clean exam question.

Throughput modes

MODE	BEHAVIOUR
On-demand (default)	Pay per read and write. No forecasting. Absorbs unpredictable traffic
Provisioned	You specify read and write capacity. Cheaper at steady, predictable load
On-demand metrics	ConsumedReadRequestsUnits, ConsumedWriteRequestsUnits
Provisioned metrics	ConsumedReadCapacityUnits, ConsumedWriteCapacityUnits

ON THE EXAM

Same logic as DynamoDB. Unpredictable traffic means on-demand; steady and forecastable means provisioned.

The offline store

- **Parquet in Amazon S3**, holding the full history of every record.
- Queryable with **Amazon Athena**, and registered in the Glue Data Catalog.
- This is what you build training sets from, and what batch inference reads.
- Feature Store adds **write_time** and **api_invocation_time** columns on write.
- **write_time is what makes time-travel queries possible.** Do not treat it as noise.

> ON THE EXAM
>
> Note where it lives: your S3 bucket, in Parquet, in your catalogue. Nothing is locked away.

Joining several feature groups together

- Real models need features from **several entities**: customer, product, session.
- **DatasetBuilder** joins feature groups on a base group, defaulting to an inner join.
- Point-in-time correctness applies **across the whole join**, not just the base group.
- You can bound the result by write time, or take the N most recent records per identifier.
- The output is a dataframe or a CSV in S3, ready to hand to a training job.

> ON THE EXAM
>
> An inner join silently drops rows where one group has no record. Check the row count against the label count.

Feature processing and scheduled pipelines

- Feature Store stores values. **Something else has to compute them on a schedule.**
- **Feature processing** lets you define the transformation and run it as a managed job.
- Those jobs can be scheduled, and appear as steps in **SageMaker Pipelines**.
- Stale features are a silent failure: the endpoint keeps serving last week's values.
- Nothing alarms when ingestion stops. Monitor write metrics, not just read latency.

> ON THE EXAM
>
> An online store with no recent writes still answers every request, cheerfully, with old data.

Deletes, and what they actually do

- **DeleteRecord** removes the record from the online store immediately.
- **The offline store is append-only.** A delete writes a tombstone; history remains.
- That is deliberate. Reproducibility needs the past to stay as it was.
- DatasetBuilder can include or exclude deleted records when building a dataset.
- For a right-to-erasure request, the offline store in S3 must be handled separately.

> ON THE EXAM
>
> Append-only history and data deletion obligations pull in opposite directions. Know that before an auditor asks.

AWS Glue or Apache Iceberg?

FORMAT	BEHAVIOUR
AWS Glue	The default. Plain Parquet files registered as a Glue table
Apache Iceberg	Open table format. Supports compaction of small files
Iceberg partitioning	By event time, into daily partitions. Not configurable
AWS guidance	In most use cases, use the Apache Iceberg table format

ON THE EXAM

Streaming ingestion produces many small files. Iceberg compaction is the documented fix, and it runs without blocking reads or writes.

A point-in-time query asks: what did we know about this customer on that day?

What a point-in-time join prevents

- Naive approach: join today's feature values onto historical labels. Everything leaks.
- A customer who churned in March gets February features computed from June data.
- A point-in-time join returns the value **as it stood at the label's timestamp**.
- Enabled through **DatasetBuilder** with point_in_time_accurate_join.
- Related options: include deleted records, include duplicates, take N most recent per identifier.

> ON THE EXAM
>
> This is temporal leakage from module 05, solved by a service rather than by remembering to be careful.

Batch and streaming ingestion

- **Batch:** a Processing or Glue job computes features, then writes with PutRecord.
- **Streaming:** a Lambda or Flink consumer writes as events arrive.
- Streaming keeps online features fresh, which matters for session and recency features.
- Both paths use the same API, so a feature group can take both at once.
- Streaming into a Glue-format offline store creates many small files. Iceberg is the answer.

> ON THE EXAM
>
> This is where module 03's Flink returns: rolling aggregates computed in the stream, written to the online store.

Feature Store, or just Parquet in S3?

FEATURE STORE, WHEN

Skew or reuse is real

- A real-time endpoint needs the same features.
- Several models share feature definitions.
- Point-in-time correctness is needed.
- Lineage and governance are required.

S3 AND PARQUET, WHEN

Neither problem exists

- One model, one team, batch only.
- Features are recomputed from scratch each run.
- No online serving path at all.
- The pipeline is already simple and works.

THE TIEBREAKER

Feature Store earns its cost when two consumers share a definition. With one consumer it is machinery without a problem.

Security and lineage

- **KMS encryption** configurable separately for the online and offline stores.
- IAM controls who can create groups, write records and read them.
- **Keep PII in its own feature group**, so access can be restricted without splitting everything.
- Lineage links feature groups to the models trained from them.
- Integrates with SageMaker Pipelines for orchestrated ingestion, and with the Model Registry.

> ON THE EXAM
>
> Feature-level access control does not exist. Group-level does. Design your groups with that in mind.

What Feature Store costs you

COMPONENT	HOW IT BILLS
Online store, on-demand	Per read request unit and write request unit
Online store, provisioned	Per provisioned read and write capacity, whether used or not
Online store storage	Per GB-month held. InMemory is the premium tier
Offline store	S3 storage, plus Athena charges when you query it
Ingestion	Whatever compute wrote the record: Glue, Processing, Lambda, Flink

ON THE EXAM

An online store provisioned for a model that only runs in batch is a recurring bill for a capability nobody calls.

Quotas worth knowing about

- Feature groups per account, and features per group, are both quota-limited.
- **InMemory default maximum is 50 GiB** per feature group.
- Provisioned capacity **decreases** are capped at roughly 27 per day.
- Quotas are visible in Service Quotas, and most can be raised on request.
- Design around entities rather than models, and you will hit group limits far more slowly.

> ON THE EXAM
>
> 'We hit a limit' is rarely the exam answer on its own. Knowing which limit, and that it is raisable, usually is.

Sharing features between accounts

- The offline store is S3 and Glue tables, so **normal cross-account sharing applies**.
- **Lake Formation** can govern access to the offline store tables centrally.
- The online store is accessed through the Feature Store runtime API, not through S3.
- Cross-account online reads mean IAM role assumption, plus PrivateLink if traffic must stay private.
- A central feature-engineering account with consuming ML accounts is the common pattern.

> ON THE EXAM
>
> The two stores have different sharing models because they are different technologies underneath. Do not assume one answer covers both.

Symptoms, and what they usually mean

SYMPTOM	USUAL CAUSE
Production accuracy below validation, no code change	Features still computed separately for inference
Training set looks too good	No point-in-time join. Current values joined to old labels
Athena queries over the offline store are slow	Many small files from streaming. Move to Iceberg
Online reads throttling intermittently	InMemory tier scaling, or provisioned capacity too low
No offline history despite writing records	Feature group is online-only, or on the InMemory tier
Endpoint latency rose after adding Feature Store	An extra network call per prediction. Budget for it

ON THE EXAM

The last one is worth planning for. Fetching features adds a hop, and that hop is inside your latency budget.

Write the feature **once**. Everything else Feature Store does follows from that.

A lender's credit model computes fourteen aggregate features. For training, a Glue job computes them from the data warehouse. For live scoring, the loan application service recomputes them in Java before calling the endpoint.

Production approval rates differ measurably from validation. An audit finds three of the fourteen features are calculated differently in the two paths.

What should the team implement?

A Add unit tests asserting the Java and Spark implementations agree on sample data

B Write the features to a Feature Store group with online and offline stores enabled

C Move the Java feature code into a Lambda function shared by both paths

D Retrain the model on features produced by the Java implementation instead

◆ PAUSE AND DECIDE

What should the team implement?

A. Add unit tests asserting the Java and Spark implementations agree on sample data

Detects divergence on the cases you thought to test. Two implementations remain, and they will drift again on the case you did not.

B. Write the features to a Feature Store group with online and offline stores enabled

One write, two readers. Training reads the offline store, the endpoint reads the online store, and the second implementation disappears.

C. Move the Java feature code into a Lambda function shared by both paths

Genuinely reduces to one implementation, then asks a training job over the warehouse to call Lambda per row. Correct instinct, wrong mechanism.

D. Retrain the model on features produced by the Java implementation instead

Picks a winner between two implementations and keeps both. The divergence returns the next time either side changes.

THE SIGNAL

The test for every option: does exactly one implementation survive? Only B removes the second one rather than policing it.

A gaming company needs sub-millisecond feature retrieval for a real-time matchmaking model handling very high request rates. An engineer proposes the InMemory online store tier.

The same features must also be available historically, so the model can be retrained monthly on the last two years of matches.

What is the problem with this design?

A The InMemory tier does not support provisioned throughput mode

B The InMemory tier cannot use Apache Iceberg for its offline store

C Nothing. Enable online and offline stores on an InMemory feature group

D The InMemory tier supports online-only groups, so there is no offline store to retrain from

◆ PAUSE AND DECIDE

What is the problem with this design?

A. The InMemory tier does not support provisioned throughput mode

Throughput mode is not what blocks this design, and it would not stop historical retraining even if it were true.

B. The InMemory tier cannot use Apache Iceberg for its offline store

Names a real concept in the wrong place. The issue is not which table format the offline store uses; it is that there is no offline store.

C. Nothing. Enable online and offline stores on an InMemory feature group

The configuration the engineer wants, and the one the service does not offer. InMemory groups are online-only.

D. The InMemory tier supports online-only groups, so there is no offline store to retrain from

Documented limitation. No replication to an offline store means no history, so monthly retraining has nothing to read.

THE SIGNAL

The fix is the Standard tier with online and offline enabled, accepting single-digit millisecond latency instead of sub-millisecond.

A subscription business builds a churn training set by querying the Feature Store offline store for each customer's current feature values, then joining them to historical churn labels going back two years.

The resulting model scores 0.94 AUC in validation and performs close to random in production. No feature obviously encodes the outcome.

What is the most likely cause?

A The join used current feature values rather than a point-in-time accurate join

B The offline store is using the Glue table format instead of Apache Iceberg

C The online store TTL expired records needed for training

D The feature group uses on-demand throughput and dropped writes under load

◆ PAUSE AND DECIDE

What is the most likely cause?

A The join used current feature values rather than a point-in-time accurate join

A label from eighteen months ago paired with today's features. Every row carries information from after the event it predicts.

B The offline store is using the Glue table format instead of Apache Iceberg

Affects query speed and small-file counts, not correctness. Both formats return the same values.

C The online store TTL expired records needed for training

Training reads the offline store, which TTL does not touch. TTL applies to online records only.

D The feature group uses on-demand throughput and dropped writes under load

On-demand absorbs load rather than dropping writes, and missing data would lower validation scores rather than inflate them.

THE SIGNAL

The phrase 'current feature values joined to historical labels' is the entire tell. It describes temporal leakage precisely.

A research team has one model, retrained monthly, scored in a nightly batch job. Features are recomputed from scratch by a single Glue job at each retrain, and the same job produces the batch scoring input.

A new architect proposes adopting SageMaker Feature Store with online and offline stores, to follow MLOps best practice.

How should you assess this?

A Adopt it fully. Online and offline stores are the standard production pattern

B Adopt it, but use the InMemory tier to keep online costs low

C Push back. There is no online serving path and one consumer, so the online store adds cost without solving a problem

D Adopt only the online store, since the offline store duplicates data already in S3

◆ PAUSE AND DECIDE

How should you assess this?

A. Adopt it fully. Online and offline stores are the standard production pattern

Best practice for a shape this workload does not have. One job already produces both training and scoring input, so there is no skew to remove.

B. Adopt it, but use the InMemory tier to keep online costs low

InMemory is the premium low-latency tier, not the cheap one, and it would remove the offline history this batch workload needs.

C. Push back. There is no online serving path and one consumer, so the online store adds cost without solving a problem

Skew needs two implementations, and reuse needs two consumers. Neither exists. An offline-only group would still add lineage if wanted.

D. Adopt only the online store, since the offline store duplicates data already in S3

Exactly inverted. A batch workload needs history and has no real-time path, so the online store is the half that serves nothing.

THE SIGNAL

Feature Store removes skew between two implementations. With one job producing both datasets, there is no second implementation.

What to take from this module

- **Write once, read twice.** Offline for training history, online for the latest value.
- **Record identifier and event time are mandatory.** Event time enables time travel.
- **InMemory tier is online-only.** No offline store, so no history. 50 GiB default limit.
- **Point-in-time accurate joins** are how you build a training set without leaking the future.
- **Iceberg for the offline store** when streaming ingestion is producing small files.

> ON THE EXAM
>
> Module 07 covers labelling and human review: Ground Truth, Mechanical Turk and A2I, with a maintenance flag.